

C++11特性

统一的初始化方法

可以统一用{}初始化

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
using namespace std;

int main() {
    // ===== 基本类型 =====
    int a = 10;           // 传统写法
    int b{10};           // 统一初始化（推荐）

    double x{3.14};
    char ch{'A'};
    bool flag{true};

    // ===== 数组 =====
    int arr1[5] = {1, 2, 3, 4, 5}; // 传统
    int arr2[] {1, 2, 3, 4, 5};    // 统一初始化（推荐）

    // ===== STL 容器 =====
    vector<int> v1 = {10, 20, 30, 40, 50}; // 非常方便
    vector<int> v2{10, 20, 30, 40, 50};    // 省略=也可以

    set<string> s{"apple", "banana", "orange"};

    map<string, int> m{
        {"tom", 85},
        {"jack", 92},
        {"mary", 78}
    };

    cout << "统一初始化示例完成！" << endl;
    return 0;
}
```

成员变量默认初始值

```

#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name = "未命名";    // 成员变量默认初始值
    int age = 18;              // 默认值为18
    double score{85.5};        // 统一初始化写法（推荐）
    bool isMale = true;

    void print() {
        cout << "姓名: " << name
              << " 年龄: " << age
              << " 分数: " << score << endl;
    }
};

int main() {
    Student s1;                // 使用所有默认值
    s1.print();                // 输出: 姓名: 未命名 年龄: 18 分数: 85.5

    Student s2{"张三", 20, 92.5}; // 如果提供构造函数则覆盖默认值
    s2.print();

    return 0;
}

```

智能指针

类型	特点
unique_ptr	独占所有权
shared_ptr	共享所有权
weak_ptr	解决循环引用

```

#include <memory>

```

```

#include <memory>
#include <iostream>
using namespace std;

```

```
int main() {
    unique_ptr<int> p = make_unique<int>(10);

    cout << *p << endl;
}
```

```
unique_ptr<int> p1 = make_unique<int>(10);
unique_ptr<int> p2 = move(p1);
```

右值引用与移动构造函数

函数返回值为对象时，返回值对象如何初始化？

➤只写复制构造函数

return 局部对象 -> 复制
return 全局对象 -> 复制

➤只写移动构造函数

return 局部对象 -> 移动
return 全局对象 -> 默认复制
return move(全局对象) -> 移动

➤同时写 复制构造函数和 移动构造函数:

return 局部对象 -> 移动
return 全局对象 -> 复制
return move(全局对象) -> 移动

右值:

- 临时对象
- 不能取地址
- 即将被销毁

```
int b = 10;           // 10 是右值
int c = a + b;        // a+b 是右值
```

右值引用:

语法:

T&&

示例：

```
int&& x = 10;
```

移动构造函数

语法：

```
类名(类名&& other)
```

它接收：右值引用

```
#include <iostream>
using namespace std;

class MyArray {
private:
    int* data;
    int size;

public:
    // 构造
    MyArray(int s) : size(s) {
        data = new int[size];
        cout << "构造函数\n";
    }

    // 拷贝构造
    MyArray(const MyArray& other) : size(other.size) {
        data = new int[size];
        for(int i = 0; i < size; i++)
            data[i] = other.data[i];
        cout << "拷贝构造\n";
    }

    // ☒ 移动构造
    MyArray(MyArray&& other) : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
        cout << "移动构造\n";
    }

    ~MyArray() {
```

```

        delete[] data;
    }
};

```

move 函数

```
std::move(x)
```

并不会移动东西，它只是：把 x 强制转换成**右值**让**移动构造函数**被调用。

自动调用移动构造函数：

1. 返回局部对象
2. 插入容器

移动赋值运算符

```

MyArray&operator=(MyArray&& other) {
    if (this != &other) {
        delete[] data;          // 释放旧资源

        data = other.data;      // 接管资源
        other.data = nullptr;   // 置空
    }
    return *this;
}

```

lambda表达式

形式：

[外部变量访问方式说明符](参数表) -> 返回值类型

```

{
    语句组
}

```

[] 不使用任何外部变量
 [=] 以传值的形式使用所有外部变量
 [&] 以引用形式使用所有外部变量
 [x, &y] x 以传值形式使用, y 以引用形式使用
 [=,&x,&y] x,y 以引用形式使用, 其余变量以传值形式使用
 [&,x,y] x,y 以传值的形式使用, 其余变量以引用形式使用

“->返回值类型”也可以没有，没有则编译器自动判断返回值类型。 30 ...

```

#include <iostream>
using namespace std;

```

```
int main() {
    auto add = [](int a, int b) { return a + b; };

    cout << add(3, 5) << endl;        // 输出 8
    cout << add(10, 20) << endl;      // 输出 30
}
```

```
int main() {
    int a = 10, b = 20;

    auto func1 = [=]() { cout << a + b << endl; };    // 值捕获
    auto func2 = [&]() { a = 100; };                // 引用捕获，可修改外部
变量

    func1();
    func2();
    cout << a << endl;    // 输出 100
}
```

mutable关键字

```
auto f = [x]() mutable {
    x++;    //  允许修改副本，修改的只是副本，不影响外部变量
};
```